



```
[1]: import hashlib
import json
import time
from copy import deepcopy
from typing import Any
```

```
[2]: #Fuctions that are defined and will be used

def sha256_string(s: str) -> str:
    return hashlib.sha256(s.encode("utf-8")).hexdigest()
def canonical_dumps(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"))
```

```
[3]: #Question 1 A single block in the simplified Ledger.
class Block:
```

```
    def __init__(
        self,
        index: int,
        timestamp: int,
        transactions: list[dict[str, Any]],
        previous_hash: str,
        nonce: int = 0,
    ) -> None:
        self.index = index
        self.timestamp = int(timestamp)
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = int(nonce)
        self.hash = self.compute_hash()
```

```
    def payload_for_hash(self) -> dict[str, Any]:
        return {
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
        }

    def compute_hash(self) -> str:
        raw = canonical_dumps(self.payload_for_hash())
        return sha256_string(raw)

    def to_dict(self) -> dict[str, Any]:
        data = self.payload_for_hash()
        data["hash"] = self.hash
        return data
```

```
# Checking the block test
demo = Block(
    index=0,
    timestamp=1_500_000_000,
    transactions=[],
    previous_hash="0" * 64,
    nonce=0,
)
assert "hash" not in demo.payload_for_hash()
assert set(demo.payload_for_hash()) == {
    "index",
    "timestamp",
    "transactions",
    "previous_hash",
    "nonce",
}
assert demo.hash == demo.compute_hash()
assert len(demo.hash) == 64
assert demo.to_dict()["hash"] == demo.hash
print("Block class OK")
print(" hash:", demo.hash[:30], "...")
```

```
Block class OK
hash: 734f0efe19a7059657241ff28da248 ...
```

[4]: #Question 2

```
walk = Block(
    index=1,
    timestamp=1_500_000_100,
    transactions=[{"sender": "Siphosethu", "recipient": "Mba", "amount": 100, "currency": "ZAR",
                  "date": "2026-08-10", "description": "Payment for services"}],
    previous_hash=demo.hash,
    nonce=0,
)
```

```
payload = walk.payload_for_hash()
raw = canonical_dumps(payload)
digest = sha256_string(raw)
```

```
# Extract all material fields used in the hash and Compute SHA-256 hash
print("1. Payload (material fields only):")
for key, value in sorted(payload.items()):
    print(f"    {key}: {value!r}")
```

```
print("\n2. Canonical JSON string:")
print("    ", raw)
```

```
print("\n3. SHA-256 digest:")
print("    ", digest)
```

```
assert digest == walk.hash == walk.compute_hash()
print("\nCommitment walkthrough OK - stored hash matches recomputation")
```

```
1. Payload (material fields only):
   index: 1
   nonce: 0
   previous_hash: '734f0efe19a7059657241ff28da248a2d6496833ed8034bb9a4abaf7dd87827'
   timestamp: 1500000100
   transactions: [{"sender": 'Siphosethu', 'recipient': 'Mba', 'amount': 100, 'currency': 'ZAR', 'date': '2026-08-10', 'description': 'Payment for services'}]]

2. Canonical JSON string:
   {"index":1,"nonce":0,"previous_hash":"734f0efe19a7059657241ff28da248a2d6496833ed8034bb9a4abaf7dd87827","timestamp":1500000100,"transactions":[{"amount":100,"currency":"ZAR","date":"2026-08-10","description":"Payment for services","recipient":"Mba","sender":"Siphosethu"}]}

3. SHA-256 digest:
   59f7b224e9f3c3e91b563c0bd5a167db106e75cb5981c06ad8b4eb7e7cf606e6
```

Commitment walkthrough OK - stored hash matches recomputation

[5]: #Question 3

```
GENESIS_PREVIOUS_HASH = "0" * 64
```

```
def create_genesis_block(timestamp: int | None = None) -> Block:
    ts = int(time.time()) if timestamp is None else int(timestamp)
    return Block(
        index=0,
        timestamp=ts,
        transactions=[],
        previous_hash=GENESIS_PREVIOUS_HASH,
        nonce=0,
    )

genesis = create_genesis_block(timestamp=1_500_000_000)
print("=== Genesis ===")
for key, value in genesis.to_dict().items():
    display = value if key != "hash" and key != "previous_hash" else (
        f"{value[:16]}..." if isinstance(value, str) and len(value) > 16 else value
    )
    print(f"    {key}: {display}")
```

```
assert genesis.index == 0
assert genesis.transactions == []
assert genesis.previous_hash == GENESIS_PREVIOUS_HASH == "0" * 64
```

```
assert genesis.previous_hash == GENESIS_PREVIOUS_HASH == "0" * 64
assert genesis.nonce == 0
assert genesis.timestamp == 1_500_000_000
assert genesis.hash == genesis.compute_hash()
print("\ncreate_genesis_block OK")
```

```
def create_linked_block(
    previous: Block,
    transactions: list[dict[str, Any]],
    timestamp: int | None = None,
    nonce: int = 0,
) -> Block:
    """Create the next block linked to `previous`."""
    ts = int(time.time()) if timestamp is None else int(timestamp)
    return Block(
        index=previous.index + 1,
        timestamp=ts,
        transactions=transactions,
        previous_hash=previous.hash,
        nonce=nonce,
    )
```

```

sample_tx = [{"sender": "Siphosethu", "recipient": "Mba", "amount": 100, "currency": "ZAR",
              "date": "2026-08-10", "description": "Payment for services"}]
block1 = create_linked_block(
    genesis,
    sample_tx,
    timestamp=1_500_000_100,
)

print("=== Block 1 ===")
print(block1.to_dict())
print("\nlinks to genesis?", block1.previous_hash == genesis.hash)

assert block1.index == 1
assert block1.previous_hash == genesis.hash
assert block1.transactions == sample_tx
assert block1.hash == block1.compute_hash()
print("\ncreate_linked_block OK")

print("\nGenesis block and subsequent block created successfully.")
print("Block 1 is linked to the genesis block.")

```

```

=== Genesis ===
index: 0
timestamp: 1500000000
transactions: []
previous_hash: 0000000000000000...
nonce: 0
hash: 734f0efe19a70596...

create_genesis_block OK
=== Block 1 ===
{'index': 1, 'timestamp': 1500000100, 'transactions': [{'sender': 'Siphosethu', 'recipient': 'Mba', 'amount': 100, 'currency': 'ZAR', 'date': '2026-08-10', 'description': 'Payment for services'}], 'previous_hash': '734f0efe19a7059657241ff28da2488a2d6496833ed0834bb9a4abaf7dd87827', 'nonce': 0, 'hash': '59f7b224e9f3c3e91b563c0bd5a167db106e75cb5981c06ad8b4eb7e7cf606e6'}

links to genesis? True

create_linked_block OK

Genesis block and subsequent block created successfully.
Block 1 is linked to the genesis block.

```

```

[6]: from copy import deepcopy

def is_hash_valid(block: Block) -> bool:
    return block.hash == block.compute_hash()

assert is_hash_valid(genesis) is True
assert is_hash_valid(block1) is True
print("genesis valid?", is_hash_valid(genesis))
print("block1 valid? ", is_hash_valid(block1))
print("is_hash_valid OK")

genesis valid? True
block1 valid? True
is_hash_valid OK

[7]: #Question 4

print("=== Tamper demo ===")
tampered = deepcopy(block1)

stored_before = tampered.hash
print("stored hash (unchanged claim):", stored_before)

tampered.transactions[0]["amount"] = 200

recomputed = tampered.compute_hash()
print("recomputed after amount=200 :", recomputed)
print("stored == recomputed?      :", stored_before == recomputed)
print("is_hash_valid(tampered)?    :", is_hash_valid(tampered))

assert stored_before != recomputed
assert is_hash_valid(tampered) is False
# Original block1 is untouched thanks to deepcopy
assert is_hash_valid(block1) is True
assert block1.transactions[0]["amount"] == 100
print("\nTamper evidence OK - edit detected; original block1 still valid")

assert stored_before != recomputed
assert is_hash_valid(tampered) is False
# Original block1 is untouched thanks to deepcopy
assert is_hash_valid(block1) is True
assert block1.transactions[0]["amount"] == 100
print("\nTamper evidence OK - edit detected; original block1 still valid")

=== Tamper demo ===
stored hash (unchanged claim): 59f7b224e9f3c3e91b563c0bd5a167db106e75cb5981c06ad8b4eb7e7cf606e6
recomputed after amount=200 : e3a318d7d9c11f5a459182bb7b7ca0c37c8b10d5ee7528a15209d36317e0fe2c
stored == recomputed?      : False
is_hash_valid(tampered)?    : False

Tamper evidence OK - edit detected; original block1 still valid

```